

Deep Learning Intro

CNNs, RNNs, and Transfer Learning

■ DEEP LEARNING

■ LEARNING OUTCOME

By the end of this module you will understand Convolutional Neural Networks for images, Recurrent Networks for sequences, and apply transfer learning to build state-of-the-art models with minimal data.

■ Skills You Will Gain

- Understand CNN architecture: Conv2D, MaxPool, BatchNorm, Dense
- Build an image classifier with Keras CNN from scratch
- Apply transfer learning with EfficientNet/ResNet50
- Understand RNNs, LSTMs, and why they handle sequences
- Implement data augmentation to prevent overfitting
- Use ModelCheckpoint and ReduceLROnPlateau callbacks

■ Deep learning powers autonomous vehicles, ChatGPT, medical imaging, speech recognition, and real-time translation. CNNs are the foundation of computer vision applications.

SECTION 1

CNN Architecture — Layer by Layer

Layer	Purpose	Key Parameters
Conv2D	Extract local features: edges, textures, shapes, patterns	<code>filters, kernel_size=(3,3), activation='relu'</code>
MaxPooling2D	Reduce spatial size — keep dominant features	<code>pool_size=(2,2), strides=(2,2)</code>
BatchNormalization	Stabilise and accelerate training after conv layers	
Flatten	Convert 3D feature maps to 1D vector for Dense layers	
Dense	Classification head — learn which features predict classes	<code>units, activation='softmax'</code>
Dropout	Prevent overfitting in the dense classification head	<code>rate=0.3 to 0.5</code>
GlobalAveragePooling2D	Average each feature map — replaces Flatten in modern CNNs	

```
from tensorflow.keras import layers, models
def build_cnn(input_shape=(224,224,3), n_classes=10):
    return models.Sequential([ layers.Conv2D(32, (3,3), activation='relu', input_shape=input_shape),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Conv2D(64, (3,3), activation='relu'),
    layers.BatchNormalization(), layers.MaxPooling2D(2,2), layers.Conv2D(128, (3,3),
    activation='relu'), layers.MaxPooling2D(2,2), layers.GlobalAveragePooling2D(), layers.Dense(256,
    activation='relu'), layers.Dropout(0.4), layers.Dense(n_classes, activation='softmax') ])
model = build_cnn()
model.compile('adam', 'sparse_categorical_crossentropy', ['accuracy'])
```

SECTION 2

Transfer Learning

Why Transfer Learning?

Training CNNs from scratch needs millions of images and days of GPU compute. Pretrained models (EfficientNet, ResNet50, VGG16) learned general features from ImageNet (14M images, 1000 classes):
Early layers: edges, corners, blobs
Middle layers: textures, patterns
Late layers: object parts, faces, wheels
We freeze these layers and train only a new classification head on our small dataset. Result: State-of-the-art accuracy with just a few hundred images and minutes of training.

```
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, models, optimizers
# Load pretrained base (trained on ImageNet)
base = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(224,224,3))
base.trainable = False # FREEZE — don't overwrite ImageNet weights
# Add custom classification head
x = layers.GlobalAveragePooling2D()(base.output)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(n_classes, activation='softmax')(x)
model = models.Model(base.input, out)
# Phase 1: Train head only (fast, few epochs)
model.compile(optimizers.Adam(1e-3), 'sparse_categorical_crossentropy', ['accuracy'])
model.fit(train_ds, epochs=10, validation_data=val_ds)
# Phase 2: Fine-tune top layers (slow, very small LR)
base.trainable = True
for layer in base.layers[:-30]: # freeze all but last 30 layers
    layer.trainable = False
model.compile(optimizers.Adam(1e-5), 'sparse_categorical_crossentropy', ['accuracy'])
model.fit(train_ds, epochs=20, validation_data=val_ds)
```

SECTION 3

RNNs and LSTMs for Sequences

Why Standard RNNs Fail on Long Sequences

RNNs process sequences step-by-step, passing hidden state forward. Problem: Gradients vanish over long sequences — early timesteps are 'forgotten'. LSTM solution: Adds gates (Forget, Input, Output) that control what to remember and what to discard. Bidirectional LSTM: reads sequence both forward and backward — richer context for each timestep.

```
from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, GRU # Text sentiment
classification with Bidirectional LSTM model = models.Sequential([ Embedding(vocab_size, 128,
input_length=max_len), # word -> dense vector Bidirectional(LSTM(64, return_sequences=True)), #
reads forward + backward LSTM(32), layers.Dense(64, activation='relu'), layers.Dropout(0.3),
layers.Dense(1, activation='sigmoid') # binary sentiment ]) model.compile('adam',
'binary_crossentropy', ['accuracy'])
```

■ PRACTICE Q & A

Q1. What does a Conv2D filter learn?

A: A small matrix (e.g. 3x3) that slides over the image and detects a specific visual pattern — edges, corners, textures. Early layers detect simple patterns; deeper layers detect complex objects.

Q2. Why do we use MaxPooling after Conv2D?

A: Reduces spatial dimensions (height x width) while keeping the most prominent features. Makes the network translation-invariant and significantly reduces parameters and compute.

Q3. What is fine-tuning in transfer learning and how does it differ from feature extraction?

A: Feature extraction: freeze all pretrained weights, train only new head. Fine-tuning: after training head, unfreeze top layers and continue training with very small learning rate (1e-5) to adapt features.

Q4. What is data augmentation and why is it important for deep learning?

A: Random transformations (flip, rotate, zoom, crop, brightness) applied to training images. Artificially expands dataset, makes model invariant to these transforms, and prevents overfitting.

Q5. When would you use an LSTM instead of a standard RNN?

A: When sequences are long (>20 steps). Standard RNNs suffer vanishing gradients and forget early context. LSTMs use gates to selectively remember or forget information across many time steps.

■ Deep Learning Cheat Sheet

<code>Conv2D(32, (3,3), activation='relu')</code>	Convolutional layer
<code>MaxPooling2D(2,2)</code>	Spatial downsampling
<code>GlobalAveragePooling2D()</code>	Pool spatial dims — modern CNN
<code>BatchNormalization()</code>	Stabilise training
<code>Dropout(0.4)</code>	Prevent overfitting
<code>EfficientNetB0(weights='imagenet')</code>	Pretrained base model
<code>base.trainable = False</code>	Freeze pretrained weights
<code>LSTM(64)</code>	Long short-term memory
<code>Bidirectional(LSTM(64))</code>	Bi-directional LSTM
<code>Embedding(vocab, 128)</code>	Word embedding layer
<code>EarlyStopping(patience=10)</code>	Stop when val plateaus
<code>ReduceLROnPlateau(factor=0.5)</code>	Halve LR on plateau